

LAPORAN PRAKTIKUM
PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 11
(LARAVEL: INTRODUCTION)



Oleh:

Ahmad Al - Farizi 2311104054

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

A. Dasar Teori

Dalam pengembangan aplikasi web modern, efisiensi dan struktur kode menjadi faktor krusial untuk membangun aplikasi yang skalabel dan mudah dikelola. Framework hadir sebagai solusi untuk menyediakan kerangka kerja yang telah teruji, mengurangi pekerjaan yang berulang, dan menstandarkan pola pengembangan. Laravel adalah salah satu web application framework berbasis PHP yang paling populer, yang mengadopsi arsitektur Model-View-Controller (MVC).

Arsitektur MVC memisahkan aplikasi menjadi tiga komponen utama:

1. Model: Bertanggung jawab atas seluruh logika terkait data, seperti interaksi dengan database (membaca, menulis, menghapus).
2. View: Merupakan bagian yang menangani presentation logic atau tampilan yang dilihat oleh pengguna. View menerima data dari Controller untuk ditampilkan.
3. Controller: Bertindak sebagai penghubung antara Model dan View. Controller menerima request dari pengguna, memprosesnya dengan memanggil logika bisnis di Model, lalu menentukan View mana yang akan ditampilkan.

Laravel menyederhanakan pengembangan web dengan menyediakan fitur-fitur unggulan seperti Routing yang kuat untuk memetakan URL ke logika aplikasi, Blade Template Engine untuk mempermudah pembuatan tampilan yang dinamis dan terstruktur, dan sistem Controller yang terorganisir untuk mengelola logika aplikasi. Dengan menggunakan Laravel, pengembang dapat fokus pada logika bisnis tanpa perlu membangun fundamental aplikasi dari nol.

B. Tujuan

1. Mahasiswa mampu memahami konsep dan implementasi MVC menggunakan web framework Laravel.

2. Mahasiswa mampu memahami dan menerapkan Routing untuk mengatur alur aplikasi.
3. Mahasiswa mampu membuat dan mengelola View untuk menampilkan antarmuka pengguna.
4. Mahasiswa mampu menggunakan Blade Template Engine untuk mempercepat dan mempermudah pembuatan tampilan.
5. Mahasiswa mampu membuat dan menghubungkan Controller untuk mengelola logika aplikasi.

BAB II

HASIL

A. GUIDED

1. ROUTE

a. Route bawaan Laravel

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PageController;

Route::get('/', function () {
    return view('welcome');
});
?>
```

Penjelasan:

Pada saat sebuah proyek Laravel dibuat, framework ini secara otomatis menyediakan satu route awal yang didefinisikan di dalam file routes/web.php. File view tersebut berada di direktori resources/views dengan ekstensi .blade.php, yang merupakan template engine bawaan Laraveldiarahkan ke URL root (/). Ketika URL tersebut dipanggil, Laravel akan menjalankan fungsi callback yang mengembalikan sebuah tampilan (view) bernama welcome. File view tersebut berada di direktori resources/views dengan ekstensi .blade.php, yang merupakan template engine bawaan Laravel.

b. Membuat Route

```
Route::get('/beranda', function () {
    return "Halaman Beranda";
});
```

Penjelasan:

Pengembang dapat menambahkan route sesuai kebutuhan aplikasi dengan mendefinisikannya di file `routes/web.php`. Route baru dibuat menggunakan class `Route` dengan metode statis seperti `get`, `post`, dan lainnya. Pada contoh `/beranda`, Laravel akan menampilkan teks sederhana ketika URL tersebut diakses. Hal ini menunjukkan bahwa route tidak selalu harus mengarah ke view, tetapi juga dapat langsung mengembalikan respons berupa string.

c. **Route Parameter**

```
Route::get('/kendaraan/{jenis}', function ($jenis) {  
    return "Tampilkan data kendaraan dengan jenis $jenis";  
});
```

Penjelasan:

Laravel mendukung penggunaan parameter pada route untuk menangkap nilai dinamis dari URL. Parameter ini dideklarasikan menggunakan tanda kurung kurawal `{}` dan akan diteruskan ke fungsi callback sebagai variabel. Dengan mekanisme ini, satu route dapat menangani berbagai variasi data berdasarkan segmen URL, misalnya untuk menampilkan data berdasarkan jenis tertentu.

d. **Route dengan Optional Parameter**

```
Route::get('/kendaraan/{jenis?}/{merek?}',  
    function ($a = 'motor', $b = 'honda') {  
        return "Cek harga kendaraan $a $b";  
    });
```

Penjelasan:

Dalam kondisi tertentu, parameter pada route tidak selalu harus diisi. Laravel menyediakan fitur parameter opsional yang ditandai dengan tanda tanya (`?`).

Agar route tetap dapat dijalankan meskipun parameter tidak dikirimkan, variabel parameter diberi nilai default di dalam fungsi callback. Pendekatan ini mencegah error 404 dan memberikan fleksibilitas dalam pemanggilan URL.

e. **Route Parameter dengan Reg. Expression**

```
Route::get('/product/{id}', function ($id) {  
    return "Tampilkan product dengan id = $id";  
});  
  
Route::get('/product/{id}', function ($id) {  
    return "Tampilkan product dengan id = $id";  
})->where('id', '[0-9]+');
```

Penjelasan:

Secara bawaan, parameter route Laravel dapat menerima nilai apa pun. Namun, dalam praktik pengembangan aplikasi, sering kali dibutuhkan pembatasan format input. Laravel memungkinkan pembatasan ini menggunakan regular expression melalui method `where()`. Dengan cara ini, parameter seperti `id` dapat dibatasi hanya menerima angka, sehingga meningkatkan validasi URL dan mengurangi kesalahan akses data.

f. **Route Redirect**

```
Route::get('/hubungi-kami', function () {  
    return '<h1>Hubungi Kami</h1>';  
});  
Route::redirect('/contact-us', '/hubungi-kami');
```

Penjelasan:

Laravel menyediakan fitur pengalihan (`redirect`) antar route tanpa perlu menulis logika tambahan di controller. Dengan `Route::redirect`, sebuah URL

dapat langsung dialihkan ke URL lain. Teknik ini berguna untuk menjaga konsistensi alamat, mendukung perubahan struktur URL, atau menyediakan alias alamat lama ke alamat baru.

g. **Route Group**

```
Route::prefix('/admin')->group(function() {  
    Route::get('/dashboard', function() {  
        return 'Tampilkan dashboard aplikasi';  
    });  
    Route::get('/datapegawai', function() {  
        return 'Tampilkan data pegawai';  
    });  
    Route::get('/datamahasiswa', function() {  
        return 'Tampilkan data mahasiswa';  
    });  
});
```

Penjelasan

Untuk mengelola route dalam jumlah banyak, Laravel menyediakan konsep route group. Route group memungkinkan beberapa route digabungkan dalam satu kelompok dengan konfigurasi yang sama, seperti awalan URL (prefix). Pada contoh prefix /admin, seluruh route di dalam grup tersebut secara otomatis memiliki awalan yang sama, sehingga struktur URL menjadi lebih rapi dan mudah dipelihara.

h. **Route Fallback**

```
Route::fallback(function () {  
    return "Maaf, alamat tidak ditemukan";  
});
```

Penjelasan:

Route fallback digunakan sebagai penanganan terakhir ketika URL yang diakses tidak cocok dengan route mana pun yang telah didefinisikan. Dengan route ini, pengembang dapat menampilkan pesan atau halaman khusus sebagai pengganti tampilan 404 bawaan Laravel. Fitur ini meningkatkan pengalaman pengguna dengan memberikan informasi yang lebih jelas saat terjadi kesalahan akses.

i. **Route Priority**

```
Route::get('/baju/1', function () {  
    return "Baju ke-1";  
});  
  
Route::get('/baju/1', function () {  
    return "Baju saya ke-1";  
});  
  
Route::get('/baju/1', function () {  
    return "Baju kita ke-1";  
});  
  
//menggunakan route parameter  
Route::get('/baju/{a}', function ($a) {  
    return "Baju ke-$a";  
});  
  
Route::get('/baju/{b}', function ($b) {  
    return "Baju saya ke-$b";  
});  
  
Route::get('/baju/{c}', function ($c) {  
    return "Baju kita ke-$c";  
});
```

Penjelasan:

Laravel memiliki aturan prioritas dalam mengeksekusi route. Untuk route tanpa parameter, jika terdapat beberapa route dengan URL yang sama, maka route yang ditulis terakhir akan dijalankan. Namun, pada route yang menggunakan parameter, Laravel akan menjalankan route pertama yang cocok dengan pola URL. Oleh karena itu, urutan penulisan route sangat penting agar tidak terjadi konflik atau hasil yang tidak sesuai dengan yang diharapkan.

2. View

a. Views bawaan Laravel

Saat aplikasi Laravel pertama kali dijalankan, halaman utama (/) akan menampilkan sebuah view default bernama welcome. Laravel menggunakan Blade sebagai template engine, sehingga seluruh file view harus berekstensi views/welcome.blade.php. Laravel menggunakan Blade sebagai template engine, sehingga seluruh file view harus berekstensi .blade.php dan diletakkan di folder resources/views.

routes/web.php

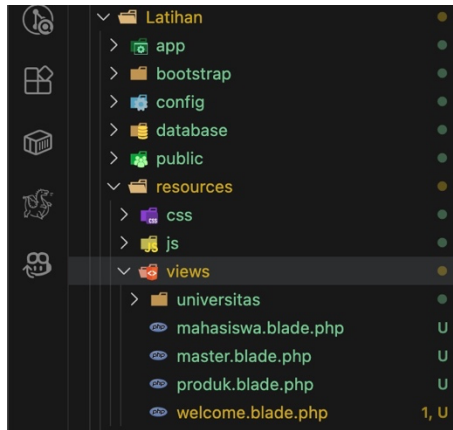
```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PageController;

Route::get('/', function () {
    return view('welcome');
});

?>
```

resources/views/welcome.blade.php



b. Membuat View

View baru dibuat dengan dua langkah utama, yaitu mendefinisikan route yang memanggil view dan membuat file view tersebut di folder resources/views. Route menggunakan perintah `Laravel::view('nama_view')`, sedangkan file view dibuat dengan nama yang sama. Ketika URL diakses, Laravel akan menampilkan isi file view tersebut.

routes/web.php

```
Route::get('/mahasiswa', function () {  
    return view('mahasiswa');  
});
```

resources/views/mahasiswa.blade.php

```
<!DOCTYPE html>  
<html>  
  <head>  
  </head>  
  <body>  
    <ol>  
      <title>Belajar Laravel</title>  
      <h2> Data Mahasiswa </h2>
```

```
<li>Mahasiswa 1</li>
<li>Mahasiswa 2</li>
<li>Mahasiswa 3</li>
<li>Mahasiswa 4</li>
</ol>
</body>
</html>
```

c. Struktur Folder View

Untuk menjaga kerapian proyek, file view dapat dikelompokkan ke dalam subfolder sesuai kebutuhan aplikasi. Jika view berada di dalam subfolder, maka pemanggilannya pada route harus menyesuaikan struktur folder tersebut. Cara ini memudahkan pengelolaan view pada aplikasi berskala besar.

routes/web.php

```
Route::get('/mahasiswa', function () {
    return view('universitas/mahasiswa');
});
```

resources/views/universitas/mahasiswa.blade.php



d. Mengirim Data ke View

Laravel memungkinkan pengiriman data dari route ke view agar tampilan bersifat dinamis. Data dapat dikirim langsung sebagai parameter kedua pada

fungsi view() atau melalui method with(). Data yang diterima di view dapat ditampilkan menggunakan variabel PHP.

- Mengirim Data ke View Sebagai Argumen

routes/web.php

```
Route::get('/mahasiswa', function () {  
    return view('universitas.mahasiswa',["mhs1"=>"Abdul Hafidz"]);  
});
```

ubah resources/views/universitas/mahasiswa.blade.php menjadi:

```
<!DOCTYPE html>  
<html>  
  <head>  
  </head>  
  <body>  
    <ol>  
      <title>Belajar Laravel</title>  
      <h2> Data Mahasiswa </h2>  
      <li><?php echo $mhs1; ?></li>  
      <li>Mahasiswa 2</li>  
      <li>Mahasiswa 3</li>  
      <li>Mahasiswa 4</li>  
    </ol>  
  </body>  
</html>
```

Mengubah data lainnya.

routes/web.php

```
Route::get('/mahasiswa', function () {  
    return view('universitas.mahasiswa',  
    [  
        "mhs1" => "Abdul hafidz",
```

```

        "mhs2" => "Aufar Bintang",
        "mhs3" => "Muhammad Nur Hamada",
        "mhs4" => "Hastin Ajeng"
    ];
});

```

ubah resources/views/universitas/mahasiswa.blade.php menjadi:

```

<!DOCTYPE html>
<html>
<head>
</head>
<body>
<ol>
<title>Belajar Laravel</title>
<h2> Data Mahasiswa </h2>
<li><?php echo $mhs1; ?></li>
<li>Mahasiswa 2</li>
<li>Mahasiswa 3</li>
<li>Mahasiswa 4</li>
</ol>
</body>
</html>

```

Membuat agar lebih rapih

routes/web.php

```

Route::get('/mahasiswa', function () {
    $arrMhs = [
        "mhs1" => "Abdul hafidz",
        "mhs2" => "Aufar Bintang",
        "mhs3" => "Muhammad Nur Hamada",
        "mhs4" => "Hastin Ajeng"
    ];
});

```

```
        return view('universitas.mahasiswa', $arrMhs);
    });
```

Jika data yang dikirim berjumlah banyak dan bersifat seragam, lebih efektif jika data dikirim dalam bentuk array dan ditampilkan menggunakan perulangan foreach. Pendekatan ini membuat kode lebih ringkas dan mudah dikembangkan.

routes/web.php

```
Route::get('/mahasiswa', function () {
    $arrMhs = ["Abdul hafidz","Aufar Bintang","Muhammad Nur
Hamada","Hastin Ajeng"];
    return view('universitas.mahasiswa',['mahasiswa' => $arrMhs]);
});
```

ubah resources/views/universitas/mahasiswa.blade.php menjadi:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<title>Belajar Laravel</title>
<h2> Data Mahasiswa </h2>
<ol>
<li><?php echo $mahasiswa[0]; ?></li>
<li><?php echo $mahasiswa[1]; ?></li>
<li><?php echo $mahasiswa[2]; ?></li>
<li><?php echo $mahasiswa[3]; ?></li>
</ol>
</body>
</html>
```

Jika data memiliki bentuk yang sama

ubah resources/views/universitas/mahasiswa.blade.php menjadi:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<title>Belajar Laravel</title>
<h2> Data Mahasiswa </h2>
<ol>
<?php foreach ($mahasiswa as $nama) {
echo "<li> $nama </li>";
} ?>
</ol>
</body>
</html>
```

- Mengirim Data ke View Menggunakan Method With

Jika data yang dikirim berjumlah banyak dan bersifat seragam, lebih efektif jika data dikirim dalam bentuk array dan ditampilkan menggunakan perulangan foreach. Pendekatan ini membuat kode lebih ringkas dan mudah dikembangkan.

routes/web.php

```
Route::get('/mahasiswa', function () {
    $arrMhs = ["Abdul hafidz","Aufar Bintang","Muhammad Nur
Hamada","Hastin Ajeng"];
    return view('universitas.mahasiswa')->with(
        'mahasiswa',
        $arrMhs);
});
```

3. Blade Template Engine

a. Menampilkan Data

Blade mempermudah penulisan kode untuk menampilkan data di dalam view. Jika pada PHP murni data ditampilkan menggunakan echo, pada Blade cukup menggunakan tanda kurung ganda {{ }}. Sintaks ini lebih singkat dan otomatis menangani output data. Selain itu, Blade menyediakan directive khusus untuk logika kondisi, perulangan, komentar, serta pengelolaan layout, sehingga penulisan tampilan menjadi lebih terstruktur dan mudah dibaca.

routes/web.php

```
Route::get('/produk', function () {
    $arrProduk = [
        "prod1" => "Televisi",
        "prod2" => "Kipas Angin",
        "prod3" => "Radio"
    ];
    return view('produk', $arrProduk);
});
```

resources/views/produk.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Data Produk</title>
</head>
<body>
Produk 1 : {{ $prod1; }}
<br>
Produk 2 : {{ $prod2; }}
<br>
Produk 3 : {{ $prod3; }}
</body>
```



```
</html>
```

Blade menyediakan versi sederhana dari struktur kontrol PHP seperti if, else, dan foreach. Dengan directive Blade, penulisan logika di view tidak lagi memerlukan tag `<?php ?>`, namun tetap menghasilkan perilaku yang sama. Hal ini membuat kode tampilan lebih bersih dan fokus pada penyajian data. contoh menggunakan if else pada `resources/views/produk.blade.php`

```
<?php
if (($nilai >= 0) and ($nilai < 50)){
    echo "Tidak lulus";
} else if (($nilai >= 50) and ($nilai <= 100)) {
    echo "Lulus";
} else {
    echo "Tidak Teridentifikasi";
}
?>
```

contoh menggunakan blade pada `resources/views/produk.blade.php`:

```
@if (($nilai >= 0) and ($nilai < 50))
    Tidak lulus
@elseif (($nilai >= 50) and ($nilai <= 100))
    Lulus
@else
    Tidak Teridentifikasi
@endif
```

b. Baris Komentar dan PHP Mode

Untuk menambahkan komentar yang tidak ikut dirender ke browser, Blade menyediakan format komentar khusus dengan tanda `{{{-- --}}`. Komentar ini hanya terbaca di sisi server dan tidak muncul pada hasil HTML.

c. Merancang Layout

Blade mendukung pembuatan template layout untuk bagian halaman yang digunakan berulang, seperti header dan footer. File layout utama berisi struktur dasar halaman dan area kosong yang ditandai dengan `@yield`. View lain dapat mewarisi layout tersebut menggunakan `@extends` dan mengisi bagian yang tersedia melalui `@section`. Pendekatan ini mengurangi duplikasi kode dan mempermudah pemeliharaan tampilan aplikasi.

resource/views/master.blade.php

```
!DOCTYPE html>
<html lang="en">
<head>
<title>@yield('title')</title>
</head>
<body>
@yield('content')
</body>
</html>
```

resource/views/produk.blade.php

```
@extends('master')
@section('title','Data Produk')
@section('content')
Produk 1 : {{ $prod1; }}
<br>
<br>
Produk 2 : {{ $prod2; }}
Produk 2 : {{ $prod3; }}
@endsection
```

Dengan adanya master layout, setiap halaman cukup mendefinisikan konten yang berbeda tanpa harus menulis ulang struktur HTML utama. Jika tampilan

umum perlu diubah, cukup mengedit file master sehingga perubahan otomatis berlaku pada seluruh halaman turunan.

4. Controller

a. Akses Controller

Dalam Laravel, route berperan sebagai penghubung antara URL yang diakses pengguna dengan proses yang dijalankan aplikasi. Ketika sebuah URL diakses, Laravel akan ke controller. Pada Laravel 8, pemanggilan controller dilakukan dengan menyebutkan class controller beserta method yang akan dijalankan. Ketika sebuah URL diakses, Laravel akan mengeksekusi method yang sesuai di dalam controller tersebut.

Laravel menyediakan file Controller.php sebagai induk dari seluruh controller. File ini berada di dalam folder `app/Http/Controllers` dan menggunakan namespace yang sesuai dengan struktur direktori. Semua controller buatan akan mewarisi class ini, sehingga dapat menggunakan fitur bawaan seperti validasi, otorisasi, dan pengelolaan request.

`routes/web.php`

```
Route::get('/', [App\Http\Controllers\PageController::class,'index']);  
Route::get('/mahasiswa',[App\Http\Controllers\PageController::class,'ta  
mpil']);
```

`app/Http/Controllers/Controller.php`

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;  
use Illuminate\Foundation\Bus\DispatchesJobs;  
use Illuminate\Foundation\Validation\ValidatesRequests;  
use Illuminate\Routing\Controller as BaseController;
```

```
class Controller extends BaseController
{
    use AuthorizesRequests, DispatchesJobs, ValidatesRequests;
}
```

b. Membuat Controller

Controller dapat dibuat langsung menggunakan editor teks dengan menempatkan file di folder `app/Http/Controllers`. Pada controller tersebut didefinisikan class yang mewarisi `Controller` dan berisi beberapa method. Setiap method merepresentasikan proses yang dapat dipanggil melalui route. Route kemudian diarahkan ke method tertentu sesuai kebutuhan halaman.

`app/Http/Controllers/PageController.php`

```
<?php
namespace App\Http\Controllers;
class PageController extends Controller
{
    public function index()
    {
        return view('welcome');
    }
    public function tampil()
    {
        return "Daftar Mahasiswa";
    }
}
```

Untuk `routes/web.php` masih sama source code nya seperti di atas.

c. Penulisan Route untuk Controller

Pada Laravel versi 8, penulisan route ke controller menggunakan referensi class secara penuh (class-based routing). Hal ini berbeda dengan versi sebelumnya yang menggunakan format string. Untuk mempermudah penulisan, namespace controller dapat di-import di bagian atas file web.php, sehingga route menjadi lebih singkat dan mudah dibaca.

app/Http/Controllers/Controller.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PageController;

Route::get('/', [App\Http\Controllers\PageController::class,'index']);
Route::get('/mahasiswa',[App\Http\Controllers\PageController::class,'tampil']);
```

d. Akses View dari Controller

Data yang dikirim dari controller dapat diolah di view menggunakan Blade directive seperti @forelse. Directive ini memudahkan proses perulangan sekaligus menangani kondisi ketika data kosong. Dengan pendekatan ini, tampilan menjadi lebih rapi dan logika presentasi tetap sederhana

app/Http/Controllers/PageController.php

```
<?php

namespace App\Http\Controllers;

class PageController extends Controller
{
    public function index()
    {
        return view('welcome');
    }
    public function tampil()
    {
```

```
$arrMahasiswa = ["Kholifahdina","Rahmat Taufik","Nita  
Fitrotunimah","Defrin Anggun Saputri"];  
return view('mahasiswa')->with('mahasiswa', $arrMahasiswa);  
}  
}
```

resource/views/mahasiswa.blade.php

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<title>Data Mahasiswa</title>  
</head>  
<body>  
<div>  
<h1>Data Mahasiswa</h1>  
<div>  
<div>  
<ol>  
<@forelse ($mahasiswa as $val)>  
<li>{{ $val }}</li>  
<@empty>  
<div>Tidak ada data...</div>  
<@endforelse>  
</ol>  
</div>  
</div>  
</div>  
</body>  
</html>
```

Laravel menerapkan pola standar untuk operasi CRUD melalui penamaan method tertentu, seperti index, store, show, dan destroy. Pola ini membantu pengembang menjaga konsistensi antara URL, HTTP method, dan fungsi yang dijalankan, sehingga struktur aplikasi lebih terorganisir dan mudah dikembangkan.

B. UNGUIDED

Soal:

1. TUGAS – Router

- Buat minimal 5 route tanpa parameter
- Buat minimal 3 route dengan parameter
- Buat minimal 3 route dengan optional parameter

Route yang dibuat nantinya akan diimplementasikan pada tugas besar di mata kuliah teori.

2. TUGAS - View

Pada pembelajaran PHP dasar kita pasti sudah mempelajari pengelolaan asset.

Pada tugas kali ini terapkan pengelolaan asset pada framework Laravel.

- Buat folder asset yang didalamnya berisi folder css, js, img.
- Buat view yang menampilkan gambar yang terletak di folder img
- Buat view yang mengakses css dan javascript dari folder css dan js.

3. TUGAS - BLADE TEMPLATE ENGINE

Kerjakan tugas berikut menggunakan blade

- Buat perulangan for untuk menampilkan bilangan 1 s.d 10
- Buat perulangan while untuk menampilkan bilangan 1 s.d 10
- Buat perulangan foreach untuk menampilkan nilai pada route berikut

```
Route::get('/mahasiswa', function () {  
    $nilai = [80,64,30,76,95];
```

```
return view('mahasiswa',$nilai);  
});
```

4. TUGAS - Controller

- a. Buat root yang mengarah ke controller
- b. Tampilkan template pada view menggunakan controller

Jawaban:

1. Router

- a. 5 route tanpa parameter

```
Route::get('/', function () {  
    return view('home');  
});  
  
Route::get('/dashboard', function () {  
    return view('dashboard');  
});  
  
Route::get('/profil-desa', function () {  
    return view('profil');  
});  
  
Route::get('/data-warga', function () {  
    return view('warga.index');  
});  
  
Route::get('/kontak', function () {  
    return view('kontak');  
});
```

- b. 3 route dengan parameter


```
Route::get('/warga/{id}', function ($id) {  
    return "Detail warga dengan ID: $id";  
});  
  
Route::get('/laporan/{tahun}', function ($tahun) {  
    return "Laporan tahun $tahun";  
});  
  
Route::get('/berita/{slug}', function ($slug) {  
    return "Berita dengan judul: $slug";  
});
```

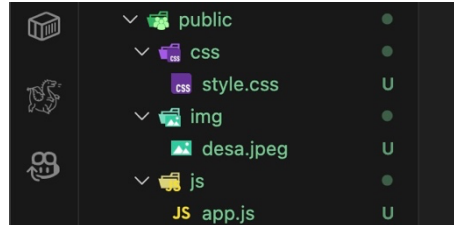
c. 3 route dengan optional parameter

```
Route::get('/login/{role?}', function ($role = 'warga') {  
    return "Login sebagai $role";  
});  
  
Route::get('/register/{tipe?}', function ($tipe = 'umum') {  
    return "Registrasi tipe $tipe";  
});  
  
Route::get('/warga/filter/{status?}', function ($status = 'aktif') {  
    return "Menampilkan warga status $status";  
});
```

2. View

- a. Buat folder asset yang didalamnya berisi folder css, js, img.

Berhubung Laravel sudah punya standar, jadi cukup pakai public.



- b. Buat view yang menampilkan gambar yang terletak di folder img
resources/view/home.blade.php

```
<!DOCTYPE html>
<html>
<head>
  <title>My Pandak</title>
</head>
<body>
  <h1>Selamat Datang di Aplikasi My Pandak</h1>

  
</body>
</html>
```

- c. Buat view yang mengakses css dan javascript dari folder css dan js.
resources/view/dashboard.blade.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
  <link rel="stylesheet" href="{{ asset('css/style.css') }}">
</head>
<body>
```

```

<h2>Dashboard Admin Desa</h2>

<script src="{{ asset('js/app.js') }}"></script>

</body>
</html>

```

3. Blade Template Engine

- a. Buat perulangan for untuk menampilkan bilangan 1 s.d 10

resources/views/mahasiswa.blade.php

```

@for ($i = 1; $i <= 10; $i++)

    <p>{{ $i }}</p>

@endfor

```

- b. Buat perulangan while untuk menampilkan bilangan 1 s.d 10

resources/views/mahasiswa.blade.php

```

@php $i = 1; @endphp

@while ($i <= 10)

    <p>{{ $i }}</p>

    @php $i++; @endphp

@endwhile

```

- c. Buat perulangan foreach untuk menampilkan nilai pada route berikut

```

Route::get('/mahasiswa', function () {
    $nilai = [80,64,30,76,95];
    return view('mahasiswa',$nilai);
});

```

routes/web.php

```

Route::get('/mahasiswa', function () {

```

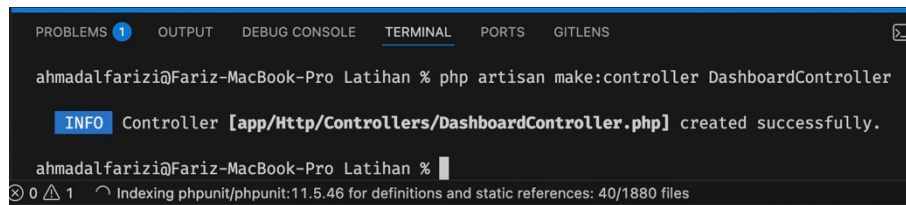
```
$nilai = [80, 64, 30, 76, 95];  
return view('mahasiswa', compact('nilai'));  
});
```

resources/views/mahasiswa.blade.php

```
@foreach ($nilai as $n)  
    <p>Nilai: {{ $n }}</p>  
@endforeach
```

4. Controller

a. Buat root yang mengarah ke controller



The screenshot shows a terminal window with the following content:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS  
ahmadalfarizi@Fariz-MacBook-Pro Latihan % php artisan make:controller DashboardController  
INFO Controller [app/Http/Controllers/DashboardController.php] created successfully.  
ahmadalfarizi@Fariz-MacBook-Pro Latihan %  
0 1 Indexing phpunit/phpunit:11.5.46 for definitions and static references: 40/1880 files
```

b. Tampilkan template pada view menggunakan controller

app/Http/Controllers/DashboardController.php

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
  
class DashboardController extends Controller  
{  
    public function index()  
    {  
        return view('dashboard');  
    }  
}
```

```
}
```

route/web.php

```
Route::get('/dashboard', [DashboardController::class, 'index']);
```

BAB III

PENUTUP

A. Kesimpulan

Melalui praktikum ini, mahasiswa telah memahami dan menerapkan dasar penggunaan framework Laravel, khususnya dalam pengelolaan routing, view, Blade Template Engine, dan controller sesuai dengan konsep arsitektur Model-View-Controller (MVC). Praktikum ini menunjukkan bahwa Laravel mampu menyederhanakan proses pengembangan aplikasi web dengan struktur kode yang rapi, terorganisir, dan mudah dikembangkan. Pemahaman ini menjadi bekal awal yang penting dalam mendukung implementasi aplikasi berbasis web serta integrasinya dengan tugas besar pada mata kuliah terkait.